Kaunas University of Technology
Faculty of Informatics

Machine Learning Algorithms

Laboratory work 1

Student

IFDU-4 Denys Suntsov

Prof.

Čalnerytė Dalia

Vaitkutė Auksė

Kaunas, 2026

TABLE OF CONTENTS

1. Introduction 3

1.1. Dataset Setup 3

1.2. Data Quality Analysis & Preprocessing 3

1.3. Linear Regression Model Implementation 3

1.4. Hypothesis Testing & Cross-Validation 3

1.5. Model Evaluation & Deployment 4

2. Dataset 4

3. Data Quality Analysis 4

3.1. Summary 4

3.2. Histograms of continuous variables 5

3.3. Data quality issues 5

3.4. Relationship between variables 6

3.5. Data preparation for linear regression 8

4. Linear regression model 8

4.1. Calculation of coefficients using method of least squares 8

4.2. Calculation of coefficients using gradient descent method 9

4.3. Comparison of coefficients 10

4.4. Modified model 11

4.5. Cross validation 11

4.6. Accuracy of models 13

4.7. Verification with scikit-learn library 13

4.8. Model 13

5. Conclusion 14

1. Introduction

Main objective is to build, evaluate, and deploy a multivariable linear regression model to predict housing prices ("Price") from the Melbourne Housing Market dataset.

1.1. Dataset Setup

- Download the Melbourne Housing Market dataset.
- If the dataset has more than 20 features, randomly select 20 features including both numerical and categorical variables.

1.2. Data Quality Analysis & Preprocessing

- Descriptive summary: calculate summary statistics for continuous variables (count, missing values %, cardinality, min, max, Q1, Q3, mean, median, std) and categorical variables (count, missing values %, cardinality, modes, mode frequencies).
- Distributions and data quality: plot histograms for continuous features, identify missing values and outliers, and justify the strategy for handling them.
- Feature relationships: explore feature relationships using scatter plots, SPLOM, or bar plots, and generate a correlation matrix for continuous variables.
- Data transformation: convert categorical variables, normalize continuous features, and create derived variables if needed. Use parameters fit only on the training split.

1.3. Linear Regression Model Implementation

- Build custom implementations from scratch for two methods:
    1. Ordinary Least Squares (OLS)
    2. Gradient Descent (or modified gradient descent)
- Display and compare the resulting coefficients from both methods side-by-side in a table.

1.4. Hypothesis Testing & Cross-Validation

- Test the statistical significance of feature coefficients and remove non-significant variables if needed.
- Run k-fold cross-validation (with k >= 5) for 3 feature subsets:
    1. All variables included
    2. Only statistically significant variables included
    3. Only the top 3 most significant variables included
- Calculate mean and standard deviation of error metrics for each case, and display all results together on a single box plot.

1.5. Model Evaluation & Deployment

- Evaluate model performance on unseen test data using error metrics and residual analysis plots.
- Compare your custom models against scikit-learn's built-in linear regression baseline.
- Prepare a deployment function/pipeline that accepts a single raw input record, applies pre-processing, and returns the target price prediction.

2. Dataset

My dataset was Melbourne Housing Market with Price column that I need to predict.

3. Data Quality Analysis

3.1. Summary

My dataset consists of 9 categorical features ('Suburb', 'Address', 'Type', 'Method', 'SellerG', 'Date', 'Regionname', 'CouncilArea', 'Postcode') and 4 numerical features ('Rooms', 'Price', 'Propertycount', 'Distance').

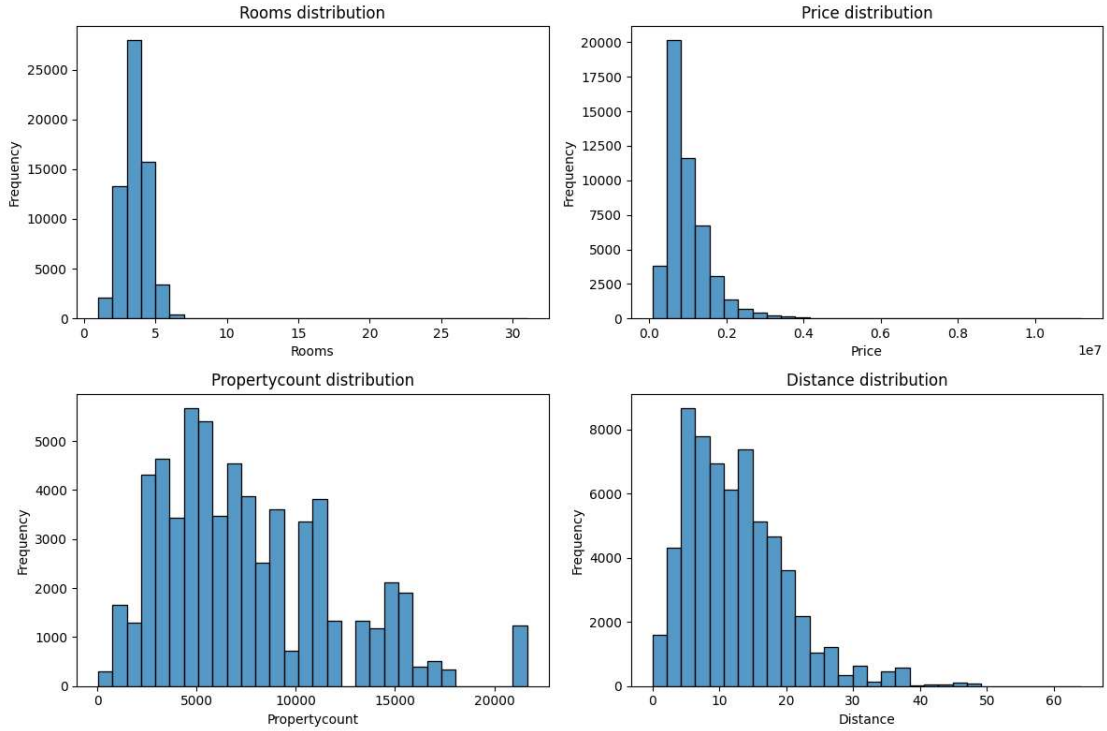
Numerical features:

	column	N	% missing	cardinality	min	Q1	median	Q3	max	mean	std
0	Rooms	63023	0.00	14	1.0	3.0	3.0	4.0	31.0	3.110595	0.957551
1	Price	63023	23.15	3417	85000.0	620000.0	830000.0	1220000.0	11200000.0	997898.241488	593498.919037
2	Propertycount	63023	0.00	368	39.0	4380.0	6795.0	10412.0	21650.0	7617.728131	4424.423167
3	Distance	63023	0.00	180	0.0	7.0	11.4	16.7	64.1	12.684829	7.592015

Categorical features:

	column	N	% missing	cardinality	mode	mode freq	mode %	mode 2	mode 2 freq	mode 2 %
0	Suburb	63023	0.0	380	Reservoir	1241	1.97	Bentleigh East	845	1.34
1	Address	63023	0.0	57754	14 Northcote St	7	0.01	5 Charles St	7	0.01
2	Type	63023	0.0	3	h	45053	71.49	u	11655	18.49
3	Method	63023	0.0	9	S	34063	54.05	PI	9790	15.53
4	SellerG	63023	0.0	476	Barry	6894	10.94	Jellis	5499	8.73
5	Date	63023	0.0	112	24/03/2018	1144	1.82	28/10/2017	1119	1.78
6	Regionname	63023	0.0	8	Southern Metropolitan	17559	27.86	Northern Metropolitan	16781	26.63
7	CouncilArea	63023	0.0	34	Boroondara City Council	5132	8.14	Darebin City Council	4182	6.64
8	Postcode	63023	0.0	225	3073	1241	1.97	3064	1093	1.73

3.2. Histograms of continuous variables



The data is mostly skewed to the right. Columns 'Rooms' and 'Price' have less obvious tails, while 'Propertycount' and 'Distance' have obvious tails.

3.3. Data quality issues

The column ‘Price’ that we want to predict has big percent of missing values, and we will drop the rows with them, because model can’t learn without a teacher.

Column ‘Rooms’ has a median of 3 rooms and an outlier of maximum of 31 rooms. I will retain such rows not to lose real data.

Columns ‘Address’, ‘Suburb’, ‘SellerG’, ‘Postcode’ and ‘CouncilArea’ show high cardinality, and I decided to drop those columns, because one-hot encoding would produce extra thousands of columns without adding generalizable pricing patterns.

Column ‘Date’ is formatted as raw string with 112 unique auction dates. We will extract ‘Year’ and ‘Month’ from it to capture temporal housing market trends as proper numerical features.

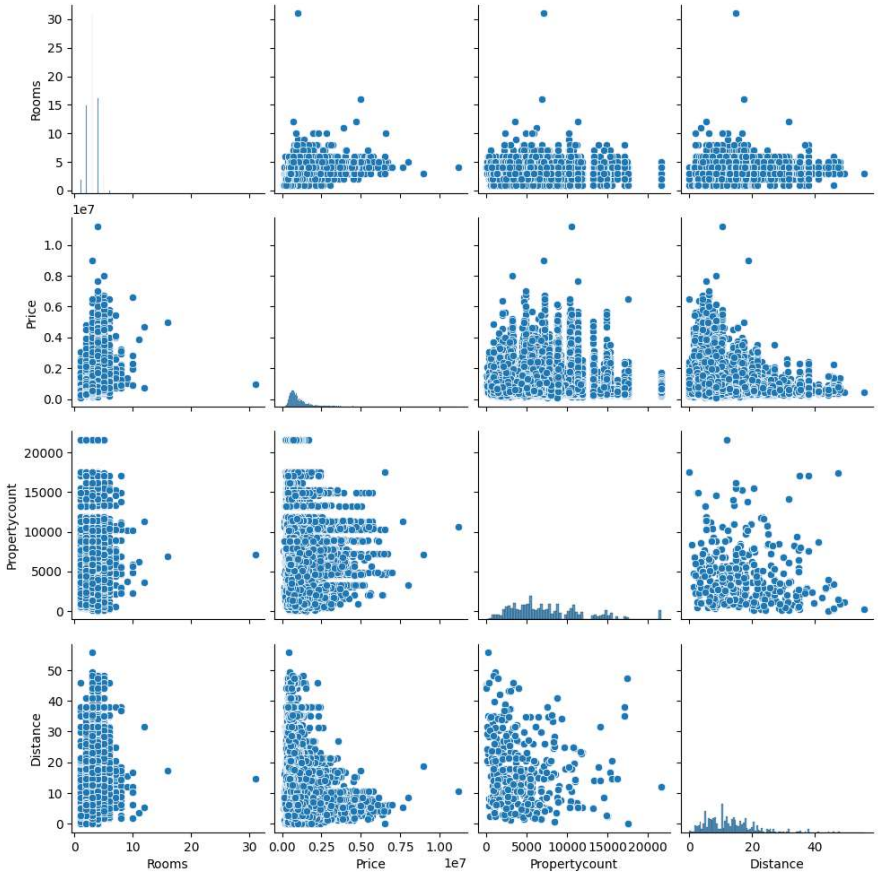
The categorical column ‘Regionname has 8 unique regions, with Southern Metropolitan (27.86%) and Northern Metropolitan (26.63%) making up more than half the dataset. We will use one-hot encoding to capture these regional price variations.

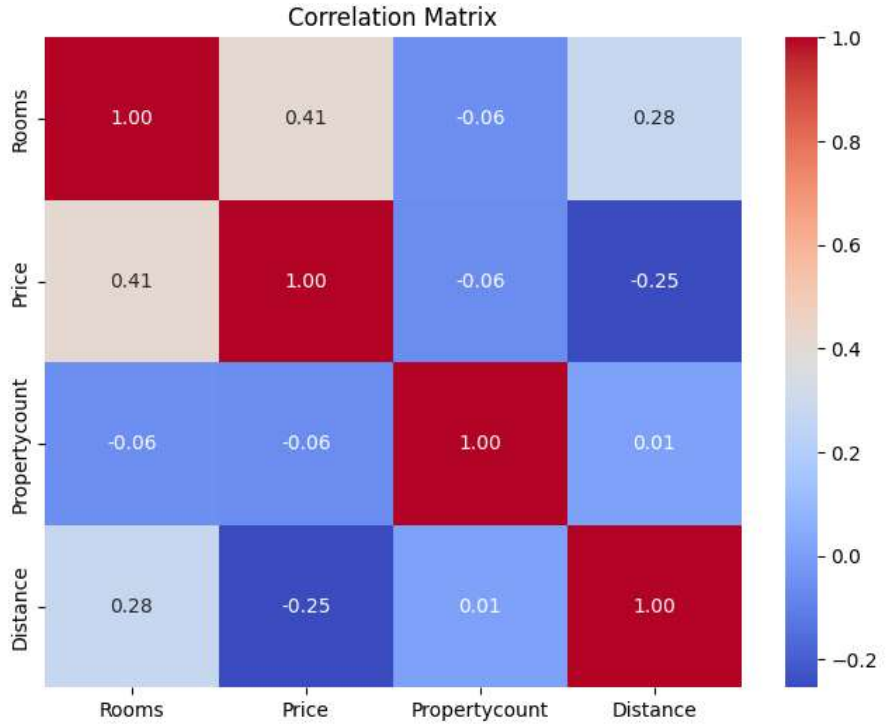
Also, 2 duplicate rows were found in the dataset, and they were removed to avoid repetiveness of data. I verified columns Price, Rooms, Distance, Propertycount for invalid values (e.g price, number of rooms etc. cannot be negative), and they were not found in dataset.

```
Price <= 0: 0
Rooms <= 0: 0
Distance < 0: 0
Propertycount < 0: 0
```

3.4. Relationship between variables

a. Scatter plot and correlation matrix:

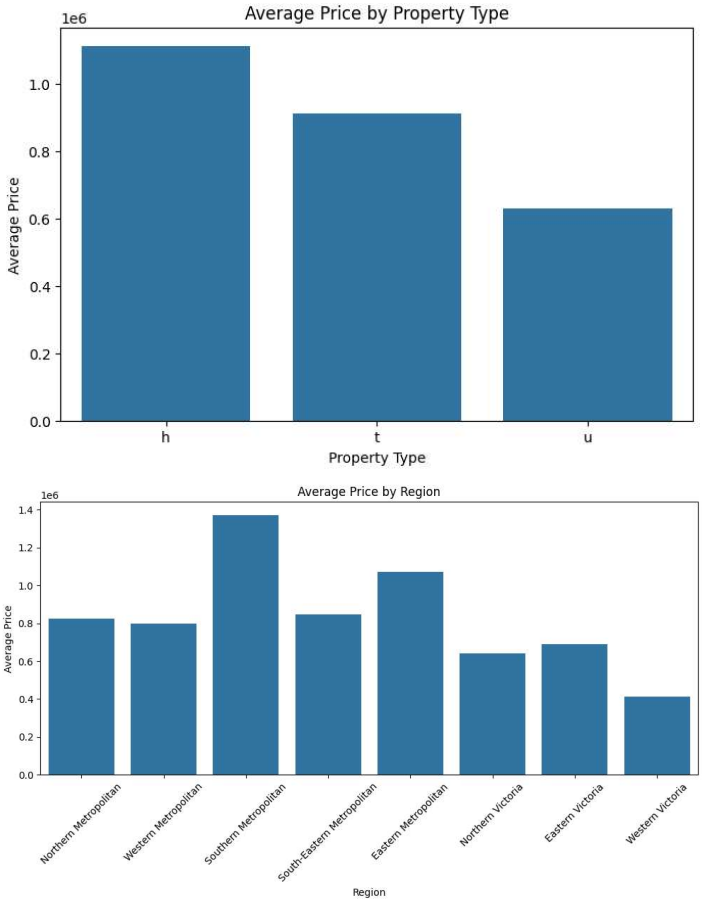





```
Pearson significance test
      Feature  Pearson r      p-value  Statistically Significant (p < 0.05)
        Rooms     0.4124 0.000000e+00                                   True
Propertycount    -0.0608 7.332001e-41                                   True
     Distance    -0.2537 0.000000e+00                                   True
```

Using scatter plot and correlation matrix, we can find good linear relationship between “Price” and “Rooms” columns (r=0.41), and p-value shows high statistical significance (0.0) of this column in prediction. “Distance” shows a moderate negative linear relationship (r=-0.25), while “Propertycount” shows weak negative linear relationship (r=-0.06). Despite such correlation results, we can see that p-value is low (7.3e-41 and 0.0 respectively), so I decided to leave them as they are significant predictors.

b. Bar plots



Bar plot of categorical columns "Type" and "Regionname" reveal significant price variations across categorical groups, e.g: "Type h" in "Type" shows higher average prices than in "Type u", and properties in "Southern Metropolitan" show higher mean prices compared to other regions.

3.5. Data preparation for linear regression

First of all, I splitted training and test data in relation 80:20. In my file, the data is ordered by 'Suburb', so I had to shuffle rows to ensure equal split in the dataset.

I created "Rooms_squared" and "Distance_squared" derived variables using formula: $x_{squared} = (x - \overline{x_{train}})^2$. If we subtract mean from variable, we will center the data around zero, and squaring creates quadratic features, which allows linear model to fit curved relationships (parabola). To prevent data leakage, during creation of squared features I use mean derived from training data.

Then, numerical features need to be standardized, so large-range columns don't dominate over low-range features: $z = \frac{x - \mu_{train}}{\sigma_{train}}$. Also, I filled missing values in those columns with medians.

As well, categorical columns need to have one-hot encoding to transfer to numerical type.

4. Linear regression model

4.1. Calculation of coefficients using method of least squares

In multivariable linear regression, we model the target variable y (house price) as a linear combination of input features $x_1, x_2, \ldots, x_k$ plus an intercept β_0:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_k x_k + \epsilon$$

To solve this efficiently for n samples and k features, we express the system in matrix notation:

$$y = X\beta + \epsilon$$

The Method of Least Squares seeks to find the coefficient vector $\hat{\beta}$ that minimizes the Sum of Squared Residuals (SSR):

$$\mathrm{SSR}(\beta) = \sum_{i=1}^{n} \left(y_i - x_i^T \beta\right)^2 = (y - X\beta)^T (y - X\beta) = (y^T - X^T \beta^T)(y - X\beta) =$$
$$= y^T y - y^T X\beta - \beta^T X^T y + \beta^T X^T X\beta = y^T y - 2\beta^T X^T y + \beta^T X^T X\beta$$

To minimize $\mathrm{SSR}(\beta)$, we take its gradient with respect to β and set it to zero:

$$\nabla_\beta \mathrm{SSR}(\beta) = -2X^T y + 2X^T X\beta = 0$$

$$-2X^T (y - X\beta) = 0$$

Solving for β gives us analytical solution:

$$\beta = (X^T X)^{-1} X^T y$$

After calculating coefficients using OLS in my dataset, I got such results:

```
Least squares coefficients:
[ 1.17220779e+06  2.52732303e+05  1.07157730e+03 -3.39887593e+05
 -4.24976032e+04  1.07842166e+05  3.70485814e+04  1.41434570e+04
 -2.13615887e+05 -4.27685641e+05  4.53317459e+04 -1.97852161e+04
  1.54772074e+04  3.50224079e+04 -9.27752186e+04 -3.25688054e+05
 -2.02635714e+05  8.41299309e+04  2.38330900e+05 -3.96199961e+05
 -2.81237346e+05]
```

4.2. Calculation of coefficients using gradient descent method

Gradient descent is iterative algorithm that finds optimal parameters by taking sequential steps down the loss function surface. The loss function that is minimized during gradient descent is Mean Squared Error:

$$J(\beta) = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2 = \frac{1}{n} (X\beta - y)^T (X\beta - y)$$

The gradient of the error surface at iteration t dictates the direction of steepest growth

$$\nabla_\beta J(\beta) = \frac{2}{n} X^T (X\beta - y)$$

Weights are updated when we move in opposite direction of the gradient. Step size is equal to learning rate:

$$\beta^{(t+1)} = \beta^{(t)} - \alpha \cdot \nabla_\beta J\left(\beta^{(t)}\right)$$

To prevent exploding gradients, I scaled the target features. As soon as gradient descent completes, the estimated coefficients β_{scaled} are rescaled back to real currency units.

After calculating coefficients using gradient descent in my dataset, I got such results:

```
Gradient descent coefficients:
[ 1.17219199e+06  2.52733414e+05  1.07369200e+03 -3.39892343e+05
 -4.24977554e+04  1.07832455e+05  3.70481356e+04  1.41432992e+04
 -2.13615129e+05 -4.27683990e+05  4.53323282e+04 -1.97753612e+04
  1.54767932e+04  3.50232814e+04 -9.26637901e+04 -3.25675790e+05
 -2.02532610e+05  8.41643293e+04  2.38341410e+05 -3.96186872e+05
 -2.80870279e+05]
```

4.3. Comparison of coefficients

When comparing the coefficients, I obtained such results:

	Variable	Least Squares	Gradient Descent	Absolute Difference
4	Rooms_squared	-4.249760e+04	-4.249776e+04	0.152194
7	Month	1.414346e+04	1.414330e+04	0.157781
12	Method_SP	1.547721e+04	1.547679e+04	0.414170
6	Year	3.704858e+04	3.704814e+04	0.445773
10	Method_S	4.533175e+04	4.533233e+04	0.582304
8	Type_t	-2.136159e+05	-2.136151e+05	0.757458
13	Method_VB	3.502241e+04	3.502328e+04	0.873455
1	Rooms	2.527323e+05	2.527334e+05	1.110377
9	Type_u	-4.276856e+05	-4.276840e+05	1.651090
2	Propertycount	1.071577e+03	1.073692e+03	2.114693
3	Distance	-3.398876e+05	-3.398923e+05	4.750382
5	Distance_squared	1.078422e+05	1.078325e+05	9.710664
11	Method_SA	-1.978522e+04	-1.977536e+04	9.854862
18	Regionname_Southern Metropolitan	2.383309e+05	2.383414e+05	10.509571
15	Regionname_Northern Metropolitan	-3.256881e+05	-3.256758e+05	12.263144
19	Regionname_Western Metropolitan	-3.962000e+05	-3.961869e+05	13.088899
0	Intercept	1.172208e+06	1.172192e+06	15.803665
17	Regionname_South-Eastern Metropolitan	8.412993e+04	8.416433e+04	34.398363
16	Regionname_Northern Victoria	-2.026357e+05	-2.025326e+05	103.104289
14	Regionname_Eastern Victoria	-9.277522e+04	-9.266379e+04	111.428553
20	Regionname_Western Victoria	-2.812373e+05	-2.808703e+05	367.066993

The coefficients estimated through those two methods are tightly aligned across all features. Core continuous predictors such as Rooms and Distance experience the difference in 1.11 and 4.75, which is equal to 0.00043% and 0.00138% respectively. The model‘s intercept error is 15.803 (0.0014%). The maximum absolute difference was found in “Regionname_Western Victoria” with absolute amount of 367.07, which is equal to 0.13%. It is because Gradient Descent terminated at a convergence treshold of 10^{-12} before reaching full convergence.

Overall, the results of the Gradient Descent differ from Least Squares method by maximum amount of 0.13%, which means that both methods are functioning properly.

4.4. Modified model

I modified model by removing columns “Method_SA”, “Propertycount” and “Method_SP”, because they were not statistically significant:

Hypothesis testing

Feature	Coefficient	Std_Err	t_stat	p_value	Significant (p < 0.05)
Intercept	1.172208e+06	7743.986439	151.370073	0.000000e+00	True
Rooms	2.527323e+05	2606.868883	96.948606	0.000000e+00	True
Propertycount	1.071577e+03	2130.913223	0.502872	6.150569e-01	False
Distance	-3.398876e+05	3128.051034	-108.657944	0.000000e+00	True
Rooms_squared	-4.249760e+04	2116.405429	-20.080086	3.141497e-89	True
Distance_squared	1.078422e+05	2880.678038	37.436383	2.430134e-301	True
Year	3.704858e+04	2221.678488	16.675942	3.234524e-62	True
Month	1.414346e+04	2199.535854	6.430201	1.289169e-10	True
Type_t	-2.136159e+05	6887.574461	-31.014676	1.233688e-208	True
Type_u	-4.276856e+05	6454.649201	-66.260091	0.000000e+00	True
Method_S	4.533175e+04	6340.119147	7.149983	8.832167e-13	True
Method_SA	-1.978522e+04	23996.123377	-0.824517	4.096508e-01	False
Method_SP	1.547721e+04	8052.534762	1.922029	5.460941e-02	False
Method_VB	3.502241e+04	8513.681017	4.113662	3.902351e-05	True
Regionname_Eastern Victoria	-9.277522e+04	25923.046658	-3.578870	3.455042e-04	True
Regionname_Northern Metropolitan	-3.256881e+05	6881.867017	-47.325537	0.000000e+00	True
Regionname_Northern Victoria	-2.026357e+05	24268.422936	-8.349769	7.063766e-17	True
Regionname_South-Eastern Metropolitan	8.412993e+04	9657.771921	8.711112	3.125945e-18	True
Regionname_Southern Metropolitan	2.383309e+05	6926.034295	34.410875	1.254391e-255	True
Regionname_Western Metropolitan	-3.962000e+05	7045.305910	-56.236020	0.000000e+00	True
Regionname_Western Victoria	-2.812373e+05	33940.951519	-8.286077	1.207635e-16	True

Dropping those columns influenced main predictors by maximum of $422 (as we dropped some columns, the difference for them is huge, because in refined model their coefficients are 0, which is normal).

Coefficient comparison

Feature	Beta_Full	Beta_Refined	Absolute_Shift
Method_SA	-1.978522e+04	0.000000e+00	19785.216088
Method_SP	1.547721e+04	0.000000e+00	15477.207413
Method_S	4.533175e+04	3.803422e+04	7297.527382
Method_VB	3.502241e+04	2.794705e+04	7075.358528
Intercept	1.172208e+06	1.178849e+06	6640.862057
Regionname_Northern Metropolitan	-3.256881e+05	-3.243058e+05	1382.284248
Propertycount	1.071577e+03	0.000000e+00	1071.577304
Regionname_Western Metropolitan	-3.962000e+05	-3.953264e+05	873.571334
Type_u	-4.276856e+05	-4.270923e+05	593.337967
Distance	-3.398876e+05	-3.394658e+05	421.805638
Regionname_Eastern Victoria	-9.277522e+04	-9.235450e+04	420.719237
Regionname_Northern Victoria	-2.026357e+05	-2.029424e+05	306.654525
Rooms	2.527323e+05	2.524476e+05	284.744400
Regionname_South-Eastern Metropolitan	8.412993e+04	8.392441e+04	205.524087
Regionname_Western Victoria	-2.812373e+05	-2.810354e+05	201.919647
Type_t	-2.136159e+05	-2.137290e+05	113.089477
Year	3.704858e+04	3.694910e+04	99.479226
Regionname_Southern Metropolitan	2.383309e+05	2.382757e+05	55.175529
Rooms_squared	-4.249760e+04	-4.244580e+04	51.801065
Distance_squared	1.078422e+05	1.078875e+05	45.326273
Month	1.414346e+04	1.415180e+04	8.347060

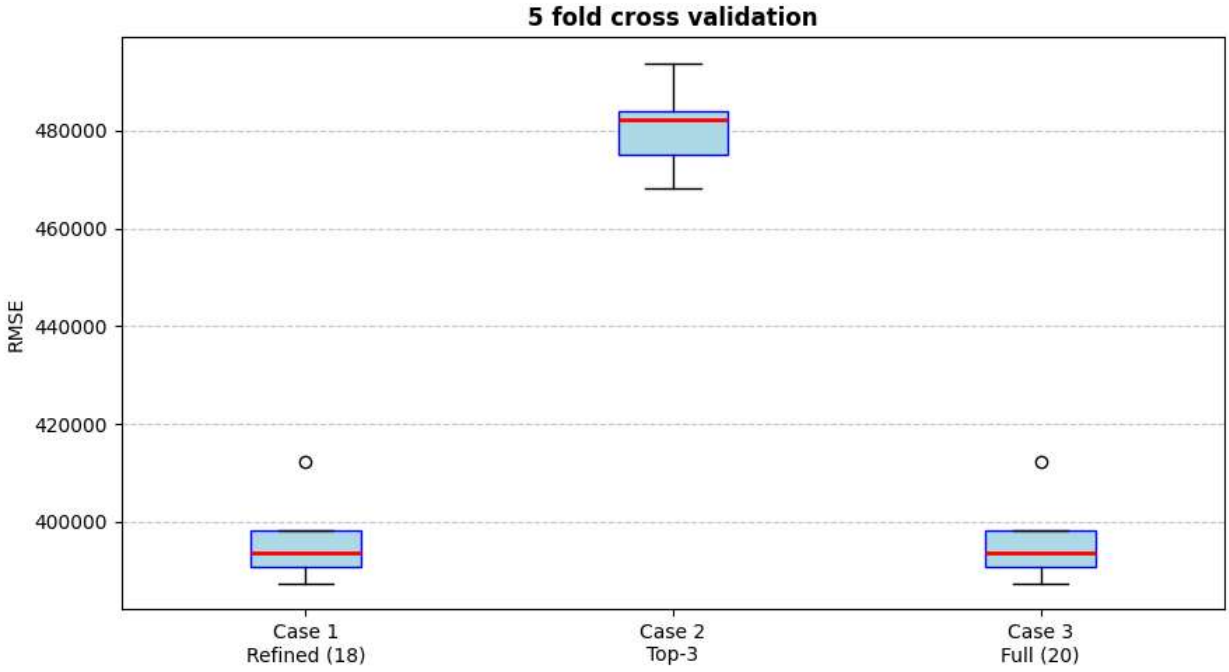
4.5. Cross validation

During k-fold cross validation, the dataset is randomly shuffled and divided into k equal-sized folds. The model runs through k total rounds, in each round: 1 fold set as the validation set, the remaining

k-1 folds are combined to serve as the training set. The model is trained on the training set and validated using the test set to check its performance. The process repeats k times so that every single fold serves as the validation set exactly once.

I created 3 types of cross validation:

- Only significant variables are included in the model
- Only 3 most significant variables are included in the model
- All possible variables are included in the model (if there were unsignificant ones)



As we can see from the boxplot, inclusion of top 3 features only increased the error, while full and refined datasets show quite equal errors:

```
5 fold cross validation results
                Model Case RMSE Mean ($) RMSE Std ($) R² Mean R² Std
Case 1: Refined model (17)   $396,529.65    $9,655.00  0.5490 0.0131
   Case 2: Top 3 model (3)   $480,620.92    $9,685.22  0.3374 0.0160
   Case 3: Full model (20)   $396,512.44    $9,660.83  0.5490 0.0131
```

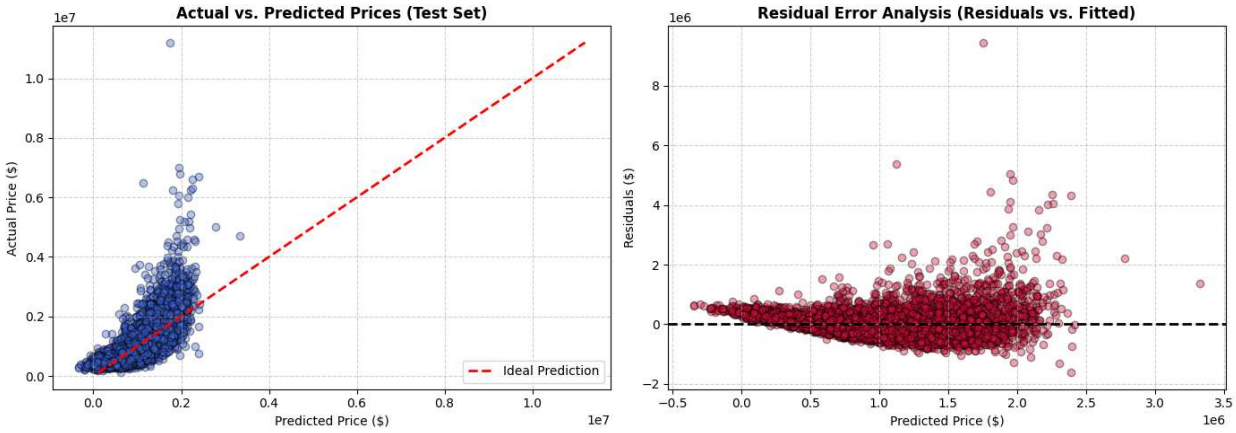
The highest error was in model that uses top 3 columns, because it cause severe underfitting, this model ignored key geographical and structural indicators. Also, this model showed the highest dispersion, which makes it less consistent across different validation splits.

Refined model and full model show almost identical predictive performance. After dropping 3 insignificant columns in case 1, we got relatively same accuracy with reduced noise. Standard deviation across 5 folds is low ($R^2 = 0.0131$), which confirms that these evaluations are not biased by a single lucky split.

4.6. Accuracy of models

I calculated accuracy for both models:

```
Test dataset performance
                Model Variant Solver     MAE ($)    RMSE ($) MAPE (%)     R²
Case 1: Refined model (17)    OLS $261,137.54 $411,020.90   29.10% 0.5391
Case 1: Refined model (17)     GD $261,137.77 $411,020.95   29.10% 0.5391
   Case 2: Top 3 model (3)    OLS $325,798.98 $492,268.43   36.03% 0.3388
   Case 2: Top 3 model (3)     GD $325,798.81 $492,268.35   36.03% 0.3388
   Case 3: Full model (20)    OLS $261,097.26 $410,997.17   29.10% 0.5391
   Case 3: Full model (20)     GD $261,097.49 $410,997.23   29.10% 0.5391
```



As expected, model with top 3 coefficients produced the biggest error across other models: R^2 drops to 0.3388 and MAE increases to $325k. Three features are not enough to capture housing prices. Full and refined models produce very similar results, so dropping 3 extra features in case 1 lost zero predictive power.

From data we can understand that there is slight difference between these two models. For properties under $2 million, model predicts price quite well, but as actual price exceeds $3 million, predictions remain around $2 million, showing underestimation for expensive houses. It is likely caused by Price skewness to the right.

As predicted house price increases, the variance of residual errors expands, showing heteroscedasticity.

4.7. Verification with scikit-learn library

The model was compared with scikit-learn library. Scikit-learn library uses OLS method to compute linear regression coefficients, so that is why my model and library's model showed similar results:

```
OLS Test MAE:   $261,137.54     GD $261,137.77     Sklearn MAE:  $261,137.54
OLS Test RMSE:  $411,020.90     GD $411,020.95     Sklearn RMSE:  $411,020.90
OLS Test MAPE:  29.10%          GD 29.10%          Sklearn MAPE:   29.10%
```

4.8. Model

I put model in the class, so it is possible to input data and get predicted price.

The model produced such output:

```
Raw record:
        Suburb: Abbotsford
        Rooms: 3
        Type: h
        Method: S
        SellerG: Biggin
        Date: 3/12/2016
        Distance: 2.5
        Postcode: 3067
        Propertycount: 4019
        Regionname: Northern Metropolitan
Predicted Melbourne House Price: $1,370,530.39
```

5. Conclusion

In this laboratory work, a complete machine learning pipeline was successfully implemented to predict housing prices using the Melbourne Housing Market dataset. Through initial data quality analysis, missing values and outliers were addressed, features were transformed, and exploratory data analysis was conducted to understand key relationships.

Two multivariable linear regression models were implemented from scratch - using Ordinary Least Squares (OLS) and Gradient Descent - and yielded comparable coefficient estimates. Statistical hypothesis testing helped refine the model by identifying the most significant features. Evaluating the feature subsets through 5-fold cross-validation demonstrated how feature selection impacts model stability, variance, and overall prediction error.

Finally, the custom model was benchmarked against the standard scikit-learn implementation, achieving similar predictive performance on unseen test data. A unified prediction function was developed to process single raw input records and output price predictions, completing the end-to-end regression pipeline.